
One ROM

One ROM CLI Manual

onerom (onerom.exe on Windows) is the command-line tool for managing One ROM ROM emulators: discovering connected devices, building and flashing firmware, inspecting device state, and manipulating ROM image files.

One ROM CLI v0.3.0

14 August 2026

Built from one-rom 32659af · source `cli-v0.3.0:docs/CLI-MANUAL.md`

Copyright © 2026 Piers Finlayson <piers@piers.rocks> · MIT licence

<https://onerom.org>

Contents

Part 1 — Guide	4
Installation	4
How One ROM talks to the CLI	5
Identifying your device	5
Common workflows	5
Program a device from a config file	5
Program from <code>--slot</code> specifications	6
Program with a plugin	6
Build firmware without flashing	6
Download and flash a pre-built release	6
Inspect a device	6
Read the live ROM image	7
Patch a running image	7
Identify a physical unit	7
Reset the host system after programming	7
See what One ROM is doing with its GPIOs	7
Erase / recover a device	8
Prepare a 16-bit ROM image	8
Device states	8
Global behaviour worth knowing	8
Part 2 — Reference	9
Synopsis	9
Global options	9
Command summary	9
scan	10
program	10
Source of the firmware (mutually exclusive groups)	11
Configuration metadata	11
Per-device overrides	11
Board, version and output	12
Reboot and flashing behaviour	12
inspect	13
inspect info	13
inspect telemetry	13
inspect slots	13
inspect image	13
inspect peek	14
inspect gpio	14
inspect header	17
inspect socket	17

control	17
control reboot.....	18
control led.....	18
control poke	18
control reset	19
control select.....	20
control pin	20
control erase	21
update	22
update slot.....	22
update commit	22
update otp	23
image.....	23
image swap-bytes.....	23
image deinterleave.....	23
image convert	24
firmware	25
firmware build	25
firmware inspect	26
firmware releases.....	26
firmware download	26
firmware chips.....	26
plugin.....	27
chips	27
Flash usage	28
board	29
board list.....	30
board header	30
board socket	30
Top-level aliases	31
peek (top-level alias)	31
poke (top-level alias).....	32
reboot (top-level alias)	32
ROM slot specification	32
Image transforms	33
Plugin specification.....	34
Pin values	35

`onerom` (`onerom.exe` on Windows) is the command-line tool for managing One ROM ROM emulators: discovering connected devices, building and flashing firmware, inspecting device state, and manipulating ROM image files.

This manual is in two parts. The **Guide** walks through installation and the common workflows. The **Reference** documents every command, subcommand and option.

This manual documents the `onerom` CLI as of release v0.3.0. Board, chip and plugin lists shown in examples are illustrative — the set your build supports may differ. Run `onerom --version` to check your version, and `onerom board list` / `onerom chips` for the definitive lists your build knows about. Commands marked **(not yet supported)** are present in the CLI surface but not yet functional.

Part 1 – Guide

Installation

Download the CLI from <https://onerom.org/cli>. Builds are provided for:

- Windows — x86 64-bit and ARM 64-bit
- macOS
- Ubuntu/Debian — x86 64-bit, and ARM 64-bit (also for Raspberry Pi)

The Windows and macOS builds are digitally signed. A sha256 checksum is published alongside every download so you can verify what you fetched.

Windows / macOS — unzip the archive and place the `onerom` (`onerom.exe`) executable in a folder on your `PATH`.

Linux — install the `.deb` as usual, which places `onerom` on your `PATH`:

```
sudo dpkg -i onerom-cli-x.y.z-1_amd64.deb
```

(replace `x.y.z` with the version, e.g. `0.1.10`, and `amd64` with `arm64` for the ARM/Pi build).

Windows SmartScreen — as a relatively new publisher, the first run may raise a *"Windows protected your PC"* dialog. Click **More info**, confirm the publisher reads *"Open Source Developer, Piers Finlayson"*, then **Run anyway**.

Verify it runs:

```
onerom --version
```

How One ROM talks to the CLI

The CLI communicates with a One ROM over USB using picoboot (the Raspberry Pi bootloader protocol, extended by `picobootx`). A One ROM is reachable in two situations:

- **Running** — normal firmware is running and serving ROMs; its USB stack (provided by the system USB plugin) exposes the `picobootx` interface.
- **Stopped** — the device is in the RP2350 bootloader (BOOTSEL). A bare RP2350 bootloader is also reachable here, which is how unprogrammed or bricked units are recovered.

Some commands work in either state; some require one specifically. Each reference entry notes when a device connection is required, and the state model is summarised under [Device states](#).

Identifying your device

With **exactly one** One ROM connected that the CLI recognises, you don't need to identify it — commands find it automatically, and the board type is inferred from the device.

With **multiple** devices connected, select one with `--serial` (`-s`). It accepts `*` and `?` wildcards:

```
onerom --serial 'A1B2*' inspect info
```

`--serial` is **global**: it can appear at any level of the command line.

A device programmed with a serial override (`program --serial-override`) reports and is matched by that overridden serial while **Running**. When **Stopped** its USB stack comes from the RP2350 bootrom, so it falls back to reporting its chip ID — match it by chip ID (or reboot it to running) in that state.

Discover what's attached:

```
onerom scan
onerom scan --slots      # also list each device's ROM slots
```

Two situations need extra flags:

- **Unrecognised / unprogrammed / bricked** units: add `--unrecognised` (`-u`) and supply `--board`, since the board type can't be inferred. The unit must still expose a valid picoboot USB interface.
- **Non-standard USB IDs**: add `--vid-pid <VID:PID>` (hex), repeatable. When supplied, only the given VID/PID pairs are matched.

`--board` (`-b`) can also be given on most commands to **override** the detected board type.

Common workflows

Program a device from a config file

The primary workflow. Build firmware from a JSON config and flash it in one step:

```
onerom program --config c64.json
```

`program` builds *and* flashes. To build a firmware binary **without** flashing, use `firmware build` instead. To build and also keep the binary while flashing, add `--output` :

```
onerom program --config c64.json --out firmware.bin
```

Program from `--slot` specifications

Instead of a config file, describe each ROM slot inline. Repeat `--slot` per slot. The required chip-select lines depend on the chip type (e.g. a 2332 needs `cs1` and `cs2`):

```
onerom program --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --slot file=basic.bin,type=2364,cs1=active-low
```

The full slot spec grammar is documented under [ROM slot specification](#) — it covers CS polarity, size handling, per-slot CPU frequency/voltage, the status LED and 16-bit forcing.

Program with a plugin

Plugins masquerade as ROMs. At most one system plugin and one user plugin are supported; a user plugin requires a system plugin. The system plugin lands in slot 0, the user plugin in slot 1:

```
onerom program --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --plugin usb
```

`--plugin` may also be combined with `--config`. The plugins are inserted ahead of the config's ROM slots (which shift up accordingly), so you can add a plugin to a stock config without editing it:

```
onerom program --config c64.json --plugin usb
```

It is an error if the config already defines a plugin of its own — remove it from the config, or drop `--plugin`.

Plugin spec forms are listed under [Plugin specification](#).

Build firmware without flashing

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
```

Download and flash a pre-built release

```
onerom firmware download --version 0.6.5 --board fire-24-e --out firmware.bin
onerom program --firmware firmware.bin
```

Or in one step — `program` will download the base firmware, build in your ROMs, and flash:

```
onerom program --config c64.json --version 0.6.5
```

Inspect a device

```
onerom inspect info      # serial, name, board, MCU, firmware version, hw revision
onerom inspect slots    # ROM slots, with the active one marked
```

Read the live ROM image

Read what the device would serve for a given logical ROM address (device must be running). The top-level `peek` is an alias for `inspect peek live`:

```
onerom peek live --address 0x100 --length 64
onerom peek live --address 0 --length 8192 --output rom-image.bin
```

Patch a running image

`poke live` writes to the ROM image currently being served, at a logical ROM offset. Changes are transient — lost on reboot. The top-level `poke` is an alias for `control poke live`:

```
onerom poke live --address 0x100 --byte 0xEA
onerom poke live --address 0 --input patch.bin
```

For file patches you can write only the differing bytes, and preview first:

```
onerom poke live --input patch.bin --delta --dry-run
onerom poke live --input patch.bin --delta
```

Identify a physical unit

Make the status LED beacon so you can spot which board is which:

```
onerom control led beacon
```

Reset the host system after programming

If you have run a wire from a One ROM header pad to the reset line of the machine One ROM is installed in, `control reset` pulses that pad low and then releases it — resetting the host so it picks up the image you just flashed. Name the pad, or the MCU GPIO behind it — `onerom inspect header` shows which that is:

```
onerom program --config c64.json
onerom control reset --pin sel_c
```

The pad is typically an image-select pad whose jumper you have removed, usually `sel_c`, or an `X1 / X2` pad. The device times the pulse, so an interrupted CLI cannot leave the host held in reset. See `control reset`, and `control pin` for driving a GPIO to an arbitrary state.

See what One ROM is doing with its GPIOs

```
onerom inspect gpio
```

One row per MCU GPIO: everything that GPIO is — its ROM socket signal under the image being served, the board peripheral it drives, the header pad it surfaces on — plus direction, level, 5V tolerance, and what One ROM itself is using it for. GPIOs connected to nothing are omitted unless you pass `--all`. Useful before driving a pin — see `inspect gpio`.

Erase / recover a device

Erase flash. This is best done while stopped; by default the command reboots the device into the required state first. A fully erased unit falls back to the RP2350 bootloader and is then reprogrammed with `--unrecognised` + `--board`:

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Read `control erase` before using it while the device is running — erasing the core firmware or system plugin will take down the USB stack.

Prepare a 16-bit ROM image

16-bit ROM types (e.g. 27C400) may need their byte pairs swapped to match the order One ROM expects. Either rewrite the file first:

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

or let the build do it, leaving the source file untouched:

```
onerom program --slot file=kick.bin,type=27C400,transform=swap_bytes
```

If your image interleaves several devices in one file — a 32-bit ROM set, say — `deinterleave` extracts a single lane. See [Image transforms](#) for the full set and how they compose.

Device states

Many commands reboot the device and, by default, pause briefly afterwards to let it re-enumerate on the USB bus.

- **Running** (default reboot target) — firmware active, serving ROMs.
- **Stopped** — RP2350 bootloader (BOOTSEL); required for some flash operations.

Common controls, where a command supports them:

- `--running` (`-r`) / `--stopped` (`-p`) — choose the post-operation state.
- `--no-reboot` — leave the device as-is.
- `--fast` — skip the re-enumeration pause.
- `--msd` (`-m`) — mount the mass-storage device when rebooting into stopped mode.

Global behaviour worth knowing

- `--yes` (`-y`) auto-confirms all prompts (non-interactive use). It also suppresses the confirmation otherwise required for CPU frequencies above 150 MHz and voltages above 1.10 V in slot specs. Use with care.
- `--verbose` (`-v`) prints device-selection progress and other detail.
- `--log-level` `<LEVEL>` sets log verbosity; defaults to `warn`. Run `onerom --help` for the accepted levels.

Part 2 – Reference

Synopsis

```
onerom [GLOBAL OPTIONS] <COMMAND> [ARGS]
```

Global options

Available on every command (they are `global` in clap terms and may appear at any level).

Option	Description
<code>--serial, -s <DEVICE></code>	Select a One ROM by serial number. Required when multiple are connected; auto-selected when exactly one is present. Accepts <code>*</code> and <code>?</code> wildcards.
<code>--vid-pid <VID:PID></code> (alias <code>--id</code>)	USB vendor/product ID pair in hex (e.g. <code>1234:abcd</code>). Repeatable; when given, only these pairs are matched. Use with <code>--unrecognised</code> .
<code>--unrecognised, -u</code> (alias <code>--unrecognized</code>)	Allow management of unrecognised/unprogrammed/bricked RP2350 boards. The unit must still expose a valid picoboot USB interface. Use with caution — permits programming any attached RP2350 board.
<code>--yes, -y</code>	Auto-confirm all prompts. Also suppresses the over-limit CPU frequency/voltage confirmations.
<code>--verbose, -v</code>	Enable verbose output.
<code>--log-level <LEVEL></code>	Set log level. Defaults to warn.
<code>--version, -V</code>	Print version.
<code>--help, -h</code>	Print help. Works on any subcommand.

Most commands accept `--board (-b)` to identify or override the board type, and rely on `--serial` (global) to pick a specific device.

Ice (STM32) boards are recognised, but the CLI cannot scan, program or build firmware for them.

Where `--board` reaches a device or builds an image, naming an Ice board is an error rather than a later failure. Commands that only report hardware take them; each command's own entry below states what it accepts, and `board list` shows which boards are which.

Command summary

Command	Purpose	Device required
<code>scan</code>	Discover connected One ROMs	No
<code>program</code>	Build and flash firmware to a One ROM	Yes
<code>inspect</code>	Read-only device state and information	Yes
<code>control</code>	Transient (non-persistent) device actions	Yes

Command	Purpose	Device required
<code>update</code>	Persistent device modifications	Yes
<code>image</code>	ROM image file manipulation	No
<code>firmware</code>	Build, inspect and manage firmware binaries	Varies
<code>plugin</code>	List available plugins	No
<code>chips</code>	List supported chip types and their flash usage	No
<code>board</code>	List board types, or draw a board's pin header / socket	No
<code>peek</code>	Alias for <code>inspect peek live</code>	Yes
<code>poke</code>	Alias for <code>control poke live</code>	Yes
<code>reboot</code>	Alias for <code>control reboot</code>	Yes

scan

Discover and list connected One ROMs — serial, USB location, name, board type, MCU and loaded firmware version. With `--verbose (-v)`, each device also shows its MCU variant and chip ID.

```
onerom scan
onerom scan --board fire-24-e
onerom scan --slots
```

Option	Description
<code>--board <BOARD></code>	Only show devices matching this board type. Conflicts with <code>--list-boards</code> . Must be a Fire board — a scan cannot find an Ice board.
<code>--list-boards</code>	List the known board types, the same listing as <code>board list</code> .
<code>--slots</code> (alias <code>--slot</code>)	Also show the ROM slot contents for each device found. Conflicts with <code>--list-boards</code> .

Device required: no.

program

Build a firmware image (from a config file, inline `--slot` specs, or a supplied binary) and flash it to a connected One ROM. This is the primary workflow. `onerom firmware program` is an alias for this command.

```

onerom program --config c64.json
onerom program --serial '5*' --config c64.json
onerom program --board fire-24-e \
    --slot file=kernal.bin,type=2364,cs1=active-low \
    --slot file=basic.bin,type=2364,cs1=active-low
onerom program --firmware firmware.bin
onerom program --config c64.json --out firmware.bin

```

Source of the firmware (mutually exclusive groups)

Option	Description
<code>--config</code> , <code>-j <FILE></code> (aliases <code>--config-file</code> , <code>--config-json</code> , <code>--json</code>)	ROM configuration JSON file. Conflicts with <code>--slot</code> , <code>--config-name</code> , <code>--config-description</code> , <code>--save-config</code> , <code>--no-config</code> , <code>--firmware</code> .
<code>--slot <SPEC></code> (alias <code>--rom</code>)	ROM slot specification; repeatable. See ROM slot specification . Conflicts with <code>--config</code> , <code>--no-config</code> , <code>--firmware</code> .
<code>--firmware <FILE></code> (alias <code>--fw</code>)	Flash a pre-built complete firmware binary directly. Conflicts with <code>--config</code> , <code>--slot</code> , <code>--base-firmware</code> and <code>--plugin</code> because a pre-built firmware already contains all ROMs/plugins. Also conflicts with <code>--version</code> .
<code>--base-firmware <FILE></code>	Use a local minimal firmware instead of downloading. With <code>--slot</code> , ROMs are built into it; alone, requires <code>--no-config</code> . Must be built with <code>EXCLUDE_METADATA=1</code> and <code>ROM_CONFIGS=</code> . Conflicts with <code>--firmware</code> , <code>--version</code> .
<code>--no-config</code>	Confirm flashing a base firmware with no ROM configuration. Only valid with <code>--config-name</code> and/or <code>--config-description</code> . Conflicts with <code>--config</code> , <code>--slot</code> , <code>--firmware</code> , and the config-override options below.

Configuration metadata

Option	Description
<code>--plugin <SPEC></code>	Plugin specification; repeatable. See Plugin specification . May be combined with <code>--config</code> : the plugins are inserted ahead of the config's ROM slots (which shift up), and it is an error if the config already defines a plugin of its own. Conflicts with <code>--firmware</code> .
<code>--config-name <NAME></code> (alias <code>--name</code>)	Name for the generated ROM configuration. Conflicts with <code>--config</code> .
<code>--config-description <DESC></code> (aliases <code>--desc</code> , <code>--description</code>)	Description for the generated configuration. Defaults to <i>"Created by the One ROM CLI"</i> . Conflicts with <code>--config</code> .
<code>--save-config <FILE></code>	Save the generated configuration to JSON. Only valid with <code>--slot</code> or <code>--no-config</code> . Conflicts with <code>--config</code> .

Per-device overrides

These are rejected with `--no-config`.

Option	Description
<code>--instance-name <NAME></code> (aliases <code>--onerom</code> , <code>--one-rom</code> , <code>--onerom-name</code> , <code>--one-rom-name</code> , ...)	Give this One ROM a name.
<code>--serial-override <NEW SERIAL></code>	Override the device's reported serial number.
<code>--logging [B00L]</code> (aliases <code>--boot-logging</code>)	Enable boot logging. Takes an optional boolean; bare flag means <code>true</code> .
<code>--disable-swd [B00L]</code> (aliases <code>--swd-disable</code>)	Shut SWD down before ROM serving starts, so debug port accesses to SRAM don't steal cycles from the serving DMAs. SWD is available for the whole of boot — including boot logging — and goes off until the next reset. Nothing is logged past that point, and plugins get no logging. This is not a debug lockout: the boot ROM runs before the One ROM firmware does, and BOOTSEL/PICOB00T are unaffected. Optional boolean; bare flag means <code>true</code> .
<code>--turbo-boot [B00L]</code>	Enable turbo boot — starts serving faster by not reading the image select jumpers, so the first non-plugin slot is always the one served. More than one non-plugin slot is refused unless <code>--force</code> is given. Optional boolean; bare flag means <code>true</code> .

Board, version and output

Option	Description
<code>--board, -b <BOARD></code>	Target board type. Inferred from the connected device if omitted.
<code>--version <VERSION></code>	Firmware version to build against. Defaults to the latest release. Conflicts with <code>--firmware</code> , <code>--base-firmware</code> .
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Also write the built firmware to this file while flashing.

Reboot and flashing behaviour

Option	Description
<code>--stopped, -p</code>	After flashing, reboot into stopped (bootloader) mode. Conflicts with <code>--running</code> .
<code>--running, -r</code>	After flashing, reboot into running mode (the default). Conflicts with <code>--stopped</code> .
<code>--no-reboot</code>	Do not reboot after flashing. Conflicts with <code>--stopped</code> .
<code>--fast</code>	Skip the re-enumeration pause after the final reboot. Conflicts with <code>--no-reboot</code> .
<code>--msd, -m</code>	Mount mass storage when rebooting into stopped mode.
<code>--verify</code>	Verify flash by reading back after programming. (not yet supported)
<code>--force, -f</code>	Continue despite non-fatal problems: assembled firmware parse errors, a board type mismatch, and config warnings such as turbo boot with more than one non-plugin ROM slot. Each is reported as a warning instead.
<code>--batch</code> (aliases <code>--multiple</code> , <code>--multi</code>)	Program multiple devices, pausing for confirmation between each. Every board is programmed with the same configuration as the first.

Option	Description
<code>--scan-slots</code>	After programming, run <code>onerom scan --slots</code> to show the result. Conflicts with <code>--fast</code> .

Device required: yes.

inspect

Read-only inspection of a connected One ROM.

```
onerom inspect <COMMAND>
```

Subcommand	Purpose	Device required
<code>info</code>	Identity and configuration	Yes
<code>telemetry</code>	Runtime telemetry (not yet supported)	Yes
<code>slots</code>	List ROM slots	Yes
<code>image</code>	Read a slot's ROM image (not yet supported)	Yes
<code>peek</code>	Read SRAM or the live ROM image	Yes
<code>gpio</code>	Show what each GPIO is and what it is doing	Yes (running)
<code>header</code>	Draw the device board's pin header	Yes
<code>socket</code>	Draw the device board's ROM socket pinout	Yes

inspect info

Show the device's serial number, user-assigned name, board type, MCU, firmware version and hardware revision. With `--verbose` (`-v`), also shows the MCU variant and chip ID.

```
onerom inspect info
onerom --serial 1234abcd inspect info
```

inspect telemetry

Access counts, timing statistics and other runtime metrics. **(not yet supported)**

Option	Description
<code>--json</code>	Output telemetry as JSON.

inspect slots

List the ROM image slots stored on the device — index, ROM type, size and description — marking the active slot. No options.

inspect image

Read (or save) the ROM image from a slot. **(not yet supported)**

Option	Description
<code>--slot <INDEX></code>	Slot index to read. Reads the active slot if omitted.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the image data to this file.

inspect peek

Read device memory. `peek memory` reads SRAM (and, in stopped state, page-aligned flash); `peek live` reads the ROM image currently being served.

```
onerom inspect peek <COMMAND>
```

inspect peek live

Read from the live ROM image at a **logical** ROM offset (starting at 0). The device must be running. Also reachable as the top-level `peek`.

```
onerom inspect peek live --address 0x100 --length 64
onerom inspect peek live --address 0 --length 8192 --output rom-image.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Logical ROM address to read from, starting at 0. Decimal or 0x hex. Default 0.
<code>--length, -l <LENGTH></code> (aliases <code>--len, --size</code>)	Number of bytes to read. Decimal or hex. If omitted, reads to the end of the live image.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the data to this file.

inspect peek memory

Read One ROM's SRAM. Most addresses reachable via PICOBOOT can be queried. In stopped state, SRAM holds no meaningful data, and flash reads must be aligned to flash page boundaries.

```
onerom inspect peek memory --address 0x20000000 --length 128
onerom inspect peek memory --address 0x10000000 --length 8192 --output flash-start.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Address to read from. Decimal or 0x hex.
<code>--length, -l <LENGTH></code> (aliases <code>--len, --size</code>)	Number of bytes to read. Decimal or hex.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the data to this file.

inspect gpio

Show, one row per MCU GPIO, what that GPIO is on this board and what One ROM is currently doing with it.

The device must be **running** with the USB system plugin: One ROM's own command handler lives in that plugin, and a stopped device is in the RP2350 bootloader where it does not exist. See [Device states](#).

```

onerom inspect gpio
onerom inspect gpio --all
onerom inspect gpio --pin gpio9
onerom inspect gpio --pin x1

```

Option	Description
<code>--pin <PIN></code>	Show only this pin, named as <code>gpio<N></code> or as a header pad (see Pin values). Conflicts with <code>--all</code> .
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--all</code>	Also list GPIOs with no function at all. By default only GPIOs connected to something are shown.

By default the table lists only the GPIOs connected to **something** — a ROM socket signal, a board peripheral or a header pad. On a 48-GPIO board a quarter of the GPIOs are connected to nothing, and listing them buries the rows worth reading; a line beneath the table says how many were omitted. `--all` lists every GPIO. Note the filter is on what the GPIO *is*, not on what the device reports using it for: the `X1 / X2` and image-select pads report `free` and are exactly what you read this table to find, so they always appear.

The number of GPIOs the device has is its own — 30 on an RP2350A, 48 on an RP2350B.

Columns:

Column	Meaning
GPIO	MCU GPIO number.
Function	Everything this GPIO is, comma-separated in a fixed order: its ROM socket signal under the image being served (A5, D3, CS1, BYTE/VPP), then the board peripheral (Status LED, RGB LED, USB VBUS, ext flash CS), then the header pad (X1, X2, SEL_A). – if the GPIO is connected to nothing.
Dir	out if the pin's output driver is enabled, in if not.
Level	The level currently on the pad, 0 or 1.
Max V	5V if the GPIO is 5V-tolerant, 3V3 if it is an RP2350 ADC pin and therefore 3.3V-only, ? if the board is not characterised.
One ROM use	What One ROM itself is using the GPIO for: free, serving (read), serving (driven) or system.

`Function` lists everything that applies rather than stopping at the first match, so a GPIO that is genuinely two things says so: on a `fire-24-f` the Status LED and the RGB LED are the same GPIO, and it reads `Status LED`, `RGB LED`. Names that would repeat are shown once — on a 32-pin board a high address line is both the socket's `A17` and the `A17` header pad, which is one net.

`Function` names only what a **GPIO** is. A header pad may carry more than the GPIO behind it — on a Fire 24/28 board the `SEL_C` and `SEL_D` pads sit on the SWCLK and SWDIO nets — but SWCLK and SWDIO are dedicated RP2350 pins with no GPIO of their own, so they do not appear here. Run `inspect header` for the pad-by-pad view, which shows every role each pad carries.

Only `Dir`, `Level` and `One ROM use` come from the device. `Function` is derived by the CLI from the board's pin map and the chip type being served: the device deliberately reports what taking a pin over would cost, never what the pin *is*. `serving (read)` pins (address, chip-select, `/BYTE`) can be driven

and released; `serving` (driven) pins (the data pins) cannot be given back without a reboot — see `control pin`.

With `--verbose` (`-v`) the table is followed by a legend restating where each column comes from, what `Dir` means and what the `3V3` / `5V` tags mean. Nothing is lost without it: the cost of taking a serving pin over is stated at the point of action by `control pin` itself.

A board revision or ROM type this build does not recognise costs the derived names, not the listing: `Function` falls back to `-` (or, for a socket pin whose chip type is unknown, `socket pin <N>`), and with no recognised board at all nothing is filtered out, since nothing can be ruled out. On a board with no pin-header descriptor, pad names come from the board's pin assignments alone and `--verbose` says so beneath the table.

On a Fire 28 (rev C) serving a 27512:

```
One ROM Fire 28 C - Firmware: v0.7.1 State: Running Serial: FC9D67248E8E8023
```

```
GPIO state · One ROM Fire 28 (rev C) · RP235xB · serving 27512
```

GPIO	Function	Dir	Level	Max V	Current use
0	D0	out	0	5V	serving (driven)
1	D1	out	0	5V	serving (driven)
2	D2	out	1	5V	serving (driven)
3	D3	out	1	5V	serving (driven)
4	D4	out	1	5V	serving (driven)
5	D5	out	1	5V	serving (driven)
6	D6	out	1	5V	serving (driven)
7	D7	out	0	5V	serving (driven)
8	X2	in	0	5V	free
9	X1	in	0	5V	free
10	CE/PE	in	0	5V	serving (read)
11	OE/VPP	in	0	5V	serving (read)
12	A14	in	0	5V	serving (read)
13	A10	in	0	5V	serving (read)
14	A11	in	0	5V	serving (read)
15	A9	in	0	5V	serving (read)
16	A8	in	0	5V	serving (read)
17	A13	in	0	5V	serving (read)
18	A15	in	0	5V	serving (read)
19	A12	in	0	5V	serving (read)
20	A7	in	0	5V	serving (read)
21	A6	in	0	5V	serving (read)
22	A5	in	0	5V	serving (read)
23	A4	in	0	5V	serving (read)
24	A3	in	0	5V	serving (read)
25	A2	in	0	5V	serving (read)
26	A1	in	0	5V	serving (read)
27	A0	in	0	5V	serving (read)
38	SEL_C	in	1	5V	free
39	SEL_D	in	1	5V	free
40	SEL_A	in	0	3V3	free
41	SEL_B	in	0	3V3	free
44	RGB LED	out	0	3V3	system
45	Status LED	out	0	3V3	system
46	USB VBUS	in	1	3V3	system

13 GPIOs with no function are hidden – use `--all` to show them.

inspect header

Draw the connected device's pin (jumper / programming) header as ASCII. The board is inferred from the device. This is the device-oriented form of `board header`; see there for what the diagram shows.

```
onerom inspect header [--board <board>]
```

Option	Description
<code>--board</code> , <code>-b</code>	Board type, overriding what the connected One ROM reports. Only needed on a One ROM whose board type this build does not recognise.

`--board` is an override, not a substitute for the device: this command draws the board of a *connected* One ROM, so one must still be present. To draw a board by name with nothing connected, use `board header`.

inspect socket

Draw the connected device's ROM socket pinout as ASCII. The board is inferred from the device. This is the device-oriented form of `board socket`.

```
onerom inspect socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
<code>--board</code> , <code>-b</code>	Board type, overriding what the connected One ROM reports.
<code>--chip-type <chip></code> , <code>-c</code>	Label pins with this ROM type's functions instead of GPIOs, and report the chip's image size on this board.
<code>--gpio</code>	Overlay the GPIO(s) onto the <code>--chip-type</code> function view (requires <code>--chip-type</code>).

As with `inspect header`, `--board` overrides the connected One ROM's reported board type rather than standing in for the device.

control

Transient actions on a connected One ROM. These affect current state but do not persist across power cycles.

```
onerom control <COMMAND>
```

Subcommand	Purpose	Device required
<code>reboot</code>	Reboot the device	Yes
<code>led</code>	Control the status LED	Yes
<code>poke</code>	Write to SRAM or the live ROM image	Yes
<code>reset</code>	Pulse a GPIO low to reset the host system	Yes (running)
<code>select</code>	Select the active ROM slot (not yet supported)	Yes

Subcommand	Purpose	Device required
<code>pin</code>	Drive a pin high, low or high-impedance	Yes (running)
<code>erase</code>	Erase flash memory	Yes

control reboot

Restart the firmware; the device re-initialises and resumes serving. By default pauses afterwards for re-enumeration. Also reachable as the top-level `reboot`.

```
onerom control reboot
```

Option	Description
<code>--stopped, -p</code>	Reboot into stopped (bootloader) state.
<code>--running, -r</code>	Reboot into running (serving) state. Default.
<code>--fast</code>	Don't pause for re-enumeration.
<code>--msd, -m</code>	Mount mass storage when rebooting into stopped mode. Conflicts with <code>--running</code> .

`--stopped` and `--running` are mutually exclusive.

control led

```
onerom control led on
onerom control led off
```

Subcommand	Description
<code>on</code>	Turn the status LED on.
<code>off</code>	Turn the status LED off.
<code>beacon</code>	Beacon the LED to identify a physical unit.
<code>flame</code>	Flame effect on the LED.

None take options. Device required: yes.

control poke

Transient writes to device memory — changes are lost on reboot. Use `update` for persistent flash writes.

```
onerom control poke <COMMAND>
```

control poke memory

Write a single byte or a binary file to SRAM at a given address. When the device is running, virtual addresses are available (e.g. `0x90000000` is the start of the live ROM image — though prefer `poke live` for that). Writing arbitrary SRAM can corrupt firmware state.

```
onerom control poke memory --address 0x20000010 --byte 0xFF
onerom control poke memory --address 0x20000000 --input patch.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Address to write to. Decimal or 0x hex.
<code>--byte <BYTE></code> (alias <code>--value</code>)	Single byte value to write. Decimal or hex.
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Write the contents of this binary file.

Exactly one of `--byte` / `--input` is required.

control poke live

Write a single byte or a binary file to the live ROM image at a **logical** ROM offset (starting at 0). Useful for patching a running ROM without reflashing. Also reachable as the top-level `poke`.

```
onerom control poke live --address 0x100 --byte 0xEA
onerom control poke live --address 0 --input patch.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Logical ROM address, starting at 0. Decimal or 0x hex. Default 0.
<code>--byte <BYTE></code> (alias <code>--value</code>)	Single byte value to write. Decimal or hex.
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Write the contents of this binary file.
<code>--delta</code> (alias <code>--deltas</code>)	Only write bytes that differ from current device content. Requires <code>--input</code> .
<code>--dry-run</code> (alias <code>--dryrun</code>)	Show what would be written without writing. Requires <code>--delta</code> .

Exactly one of `--byte` / `--input` is required.

control reset

Pulse a GPIO low, then release it, to reset the host system One ROM is installed in — useful in scripted workflows after programming a new image.

`--pin` is the pin your reset wire is soldered to, typically an image-select pad whose jumper has been removed — `sel_c` is the usual choice, as more boards have it than have X pads and it is 5V tolerant where it exists — or an X1 / X2 pad. Name it by pad (`sel_c` , `x1`) or by MCU GPIO (`gpio9`) — see [Pin values](#). `inspect header` shows which GPIO is behind each pad.

The line is only ever **driven low and then released to high impedance**. A reset net has its own pull-up and may have other drivers on it, so there is deliberately no way to drive it high. Use `control pin` if you need arbitrary states.

The **device** times the pulse, not the CLI: if this command is interrupted, the terminal closes or the cable is pulled mid-pulse, the device still releases the pin. The device's own limit is 60 seconds.

The device must be **running** with the USB system plugin — see [Device states](#).

```

onerom control reset --pin sel_c
onerom control reset --pin gpio9
onerom control reset --pin gpio9 --hold 500

```

Option	Description
<code>--pin <PIN></code>	Pin the reset wire is connected to, named as <code>gpio<N></code> or as a header pad (see Pin values). Required.
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--hold <MS></code>	Milliseconds to hold reset asserted. Decimal or 0x hex. Default 100; 0 is rejected, because a reset pulse with no end is not a reset.

If One ROM is itself using the GPIO the command is refused, naming what it is doing; `control reset` has no `--force` of its own, and the message points at `control pin --force` for the case where that is genuinely what you want. If the GPIO is not 5V-tolerant the command warns and asks for confirmation, which `--yes` answers.

```

$ onerom control reset --pin x1
Asserted reset on x1 (gpio9) for 100ms - the device times the pulse and releases the pin

```

control select

Switch the device to serving the specified slot immediately (not persistent). **(not yet supported)**

Option	Description
<code>--slot <INDEX></code>	Slot index to activate. Required.

control pin

Drive a One ROM pin high, low or high-impedance, optionally for a bounded period.

`--pin` names an MCU GPIO or a header pad (see [Pin values](#)); the command is named for what is being addressed rather than for any one spelling.

Without `--hold` the state is latched until something else changes it. With `--hold` the **device** holds the state for that many milliseconds and then applies `--then` — high impedance unless you say otherwise. As with `control reset`, the hold is timed on the device, so an interrupted CLI cannot leave a pin latched.

The device must be **running** with the USB system plugin — see [Device states](#). `inspect gpio` shows what each GPIO is and what One ROM is using it for.

```

onerom control pin --pin gpio9 --state high
onerom control pin --pin gpio9 --state low --hold 250
onerom control pin --pin gpio9 --state low --hold 250 --then high
onerom control pin --pin x1 --state 0 --hold 250
onerom control pin --pin sel_a --state z

```

Option	Description
<code>--pin <PIN></code>	Pin to drive, named as <code>gpio<N></code> or as a header pad (see Pin values). Required.

Option	Description
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--state <STATE></code>	high, low, or z (high-impedance). 1 and 0 are accepted for high and low. Required.
<code>--hold <MS></code>	Hold <code>--state</code> for this many milliseconds, then apply <code>--then</code> . Decimal or 0x hex. Omit to latch indefinitely. The device's own limit is 60 seconds.
<code>--then <STATE></code>	State to apply when <code>--hold</code> expires: high, low or z (or 1/0). Default z. Requires <code>--hold</code> .
<code>--force</code>	Drive the GPIO even though One ROM is using it for serving.

Refusals and warnings. If One ROM is itself using the GPIO, the command is refused and names what it is doing. `--force` overrides, and prints what that costs:

- a pin serving **reads** (address, chip-select, /BYTE) is reversible — serving keeps reading it, and `--state z` puts it back;
- a pin serving **drives** (a data pin) is not — forcing it takes the pin away from the PIO that drives it, and serving stays broken until the device is rebooted.

If the GPIO is not 5V-tolerant — an RP2350 ADC pin, per the board metadata, not a measurement — the command warns and asks for confirmation, which `--yes` or `--force` answers. Nothing else about the pad is checked: what is wired to it, whether a jumper is fitted and what voltage the far end sits at are yours to know.

```
$ onerom control pin --pin x1 --state low --hold 2000
Set x1 (gpio9) low for 2000ms - the device times the hold and then sets it high impedance
```

control erase

Permanently erase flash contents — firmware, metadata and ROM images. A fully erased unit boots into the RP2350 bootloader and is reprogrammed with `--unrecognised + --board`.

Best performed while stopped; by default the command reboots into the required state first. Erasing the core firmware or the system plugin while **running** takes down the USB stack (requiring manual BOOTSEL via the header pins), and large erases may cause a temporary USB drop and re-enumerate — in which case the erase likely succeeded and can be checked with `inspect peek memory`. Anything else running from flash (e.g. a user plugin) may crash during an erase.

Offsets are relative to the flash base `0x10000000`. Ranges must be 4096-aligned. Multiple ranges may be erased in one operation.

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Option	Description
<code>--all, -a</code>	Erase all flash contents.
<code>--offset <OFFSET></code>	Erase at offset(s) from the flash base. 4096-aligned; pair each with a <code>--length</code> ; repeatable. Conflicts with <code>--address</code> .

Option	Description
<code>--address <ADDRESS></code> (alias <code>--addr</code>)	Erase at absolute address(es). 4096-aligned; pair each with a <code>--length</code> ; repeatable. Conflicts with <code>--offset</code> .
<code>--length <LENGTH></code> (aliases <code>--len</code> , <code>--size</code>)	Length of each range. 4096-aligned; specify once per <code>--offset</code> / <code>--address</code> ; repeatable. Conflicts with <code>--all</code> .
<code>--no-reboot</code> , <code>-n</code>	Don't reboot before or after erasing. Risky if One ROM is accessing the range.
<code>--stopped</code> , <code>-p</code>	Reboot into stopped mode after erasing.
<code>--running</code> , <code>-r</code>	Reboot into running mode after erasing.
<code>--msd</code> , <code>-m</code>	Mount mass storage when rebooting into stopped mode. Requires <code>--stopped</code> .
<code>--fast</code>	Don't pause for re-enumeration. Requires a reboot mode.

One of `--all` / `--offset` / `--address` is required. `--stopped` and `--running` are mutually exclusive, and both conflict with `--no-reboot`.

update

Persistent modifications — these write to flash and survive power cycles.

```
onerom update <COMMAND>
```

Subcommand	Purpose	Device required
<code>slot</code>	Write a ROM image to a flash slot (not yet supported)	Yes
<code>commit</code>	Commit the live image to flash (not yet supported)	Yes
<code>otp</code>	Read/write OTP memory (not yet supported, hidden)	Yes

update slot

Write a ROM image to a flash slot; persists across power cycles. The ROM type and chip-select configuration must match the slot's existing configuration, or the slot must be empty. (**not yet supported**)

```
onerom update slot --slot 2 --image kernal.bin
```

Option	Description
<code>--slot <INDEX></code>	Flash slot index to write. Required.
<code>--image <FILE></code>	ROM image file to write. Required.

update commit

Persist the currently active RAM image to its corresponding flash slot. (**not yet supported**)

```
onerom update commit
onerom update commit --slot 2
```

Option	Description
<code>--slot <INDEX></code>	Slot to commit. Commits the active slot if omitted.

update otp

Read or write RP2350 OTP memory, including One ROM-specific USB configuration and identity data. Hidden, advanced. **OTP writes are irreversible. (not yet supported)**

Option	Description
<code>--read</code>	Read and display OTP contents. Conflicts with <code>--write</code> .
<code>--write <ROW=VALUE></code>	Write a value to an OTP row. Conflicts with <code>--read</code> .

image

ROM image file manipulation. No device connection required.

```
onerom image <COMMAND>
```

image swap-bytes

Swap adjacent byte pairs — reverses byte order within each 16-bit word throughout the image. Required for 16-bit ROM types (e.g. 27C400) when the source has the opposite byte order to what One ROM expects. The input must have an even number of bytes.

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

Option	Description
<code>--input, -i <FILE> (alias --in)</code>	Input ROM image file.
<code>--output, -o <FILE> (alias --out)</code>	Output file path.

The same operation is available during a build as `--slot transform=swap_bytes` ; see [Image transforms](#).

Device required: no.

image deinterleave

Extract one lane from an interleaved ROM image. The image contains `--stride` interleaved lanes of `--bytes` bytes each; lane `--offset` is kept and the rest discarded. Used to split a wide ROM image, distributed as a single interleaved file, into the narrower images each device needs.

The input length must be a multiple of `--bytes × --stride` ; the output is `1/--stride` of the input length.

```
# odd bytes of a 16-bit interleaved image
onerom image deinterleave --input rom16.bin --output odd.bin --offset 1 --stride 2

# byte 2 of a 32-bit interleaved image
onerom image deinterleave --input rom32.bin --output b2.bin --offset 2 --stride 4

# the upper 16-bit half of each 32-bit word
onerom image deinterleave --input rom32.bin --output hi.bin --offset 1 --stride 2 --bytes 2
```

Option	Description
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Input ROM image file.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path.
<code>--offset <N></code>	Which lane to keep. Must be less than <code>--stride</code> .
<code>--stride <N></code>	How many lanes the image interleaves. Must be at least 2.
<code>--bytes <N></code> (alias <code>--unit</code>)	Width of one lane, in bytes. Defaults to 1; use 2 to keep 16-bit words together.

The same operation is available during a build as

`--slot transform=deinterleave:<offset>/<stride>[/<bytes>]` ; see [Image transforms](#).

Device required: no.

image convert

Convert a ROM image between formats. Reads `--input` in the `--from` format and writes `--output` in the `--to` format. Formats: `binary` (aliases `bin`, `raw`) and `ihex` (Intel HEX; aliases `intel-hex`, `intel_hex`). The format set is designed to grow — further formats can be added without changing the command.

```
onerom image convert --from ihex --to binary --input rom.hex --output rom.bin
onerom image convert --from binary --to ihex --input rom.bin --output rom.hex --load-address
$E000
```

Option	Description
<code>--from <FORMAT></code>	Input format: <code>binary</code> or <code>ihex</code> .
<code>--to <FORMAT></code>	Output format: <code>binary</code> or <code>ihex</code> .
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Input ROM image file.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path.
<code>--load-address <ADDR></code>	Intel HEX load address (decimal, or <code>0x/\$</code> -prefixed hex). Only valid when one side is <code>ihex</code> ; subtracted when reading <code>ihex</code> , used as the base when writing <code>ihex</code> . Defaults to 0.

Intel HEX output uses 16-byte records with a terminating EOF record; unwritten addresses read as `0xFF` when decoding. Device required: no.

firmware

Build, inspect and manage firmware binaries. Use `program` to flash; `firmware build` produces a binary without flashing.

```
onerom firmware <COMMAND>
```

Subcommand	Purpose	Device required
<code>build</code>	Build a firmware binary from a config	No
<code>inspect</code>	Inspect a firmware binary	No
<code>releases</code>	List firmware releases	No
<code>download</code>	Download a release binary	No
<code>chips</code>	List supported chip types and their flash usage	No
<code>program</code>	Alias for <code>onerom program</code>	Yes

firmware build

Produce a flashable firmware binary for a board and MCU from a JSON config or inline `--slot` args, without flashing.

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
onerom firmware build --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --out firmware.bin
```

The configuration options mirror `program`: `--config` (`-j`), `--slot`, `--plugin`, `--config-name`, `--config-description`, `--save-config`, `--no-config`, and the per-device overrides `--instance-name`, `--serial-override`, `--logging`, `--disable-swd`, `--turbo-boot` (all rejected with `--no-config`). Build-specific options:

Option	Description
<code>--board, -b</code> <code><BOARD></code>	Target board type. Required when not inferable from a connected device.
<code>--version</code> <code><VERSION></code>	Firmware version to build against. Defaults to latest.
<code>--base-firmware</code> <code><FILE></code>	Use a local minimal firmware instead of downloading. Must be built with <code>EXCLUDE_METADATA=1</code> and <code>ROM_CONFIGS=</code> . Conflicts with <code>--version</code> .
<code>--output, -o</code> <code><FILE></code> (alias <code>--out</code>)	Output file path. Defaults to <code>onerom-<board>-<version>.bin</code> . Conflicts with <code>--path</code> .
<code>--path <DIR></code>	Output directory, using the default filename. Conflicts with <code>--output</code> .
<code>--force, -f</code>	Continue despite non-fatal problems: assembled firmware parse errors, a board type mismatch, and config warnings such as turbo boot with more than one non-plugin ROM slot. Each is reported as a warning instead.

Device required: no.

firmware inspect

Show a firmware binary's version, board type, MCU, and embedded ROM images and metadata.

```
onerom firmware inspect --firmware firmware.bin
```

Option	Description
<code>--firmware <FILE></code> (aliases <code>--fw</code> , <code>--in</code> , <code>--input</code>)	Firmware binary to inspect.
<code>--board</code> , <code>-b <BOARD></code>	Inspect the release firmware for this board type. Conflicts with <code>--firmware</code> .
<code>--version <VERSION></code>	Firmware version to inspect. Defaults to latest. Conflicts with <code>--firmware</code> .

firmware releases

List available firmware releases with supported boards and MCUs.

```
onerom firmware releases
```

Option	Description
<code>--board</code> , <code>-b <BOARD></code>	Show only releases for this board type.
<code>--all</code> , <code>-a</code>	Show all releases even if a device is attached and detected. Conflicts with <code>--board</code> .

firmware download

Download the base (ROM-less) firmware binary for a version/board/MCU.

```
onerom firmware download --version 0.6.5 --board fire-24-e --out firmware.bin
```

Option	Description
<code>--version <VERSION></code>	Version to download. Defaults to latest.
<code>--board</code> , <code>-b <BOARD></code>	Target board type. Inferred from device if omitted.
<code>--output</code> , <code>-o <FILE></code> (alias <code>--out</code>)	Output file path. Defaults to <code>onerom_<board>_<version>.bin</code> . Conflicts with <code>--path</code> .
<code>--path <DIR></code>	Output directory, using the default filename. Conflicts with <code>--output</code> .

firmware chips

List the chip types a board can emulate and the flash each one uses, or all chip types grouped by pin count. Identical to the top-level `chips`.

```
onerom firmware chips --board fire-24-e
onerom firmware chips --board fire-24-e --chip-type 2364
onerom firmware chips --all
```

Option	Description
<code>--board, -b <BOARD></code>	Show supported chips for this board. Conflicts with <code>--all</code> .
<code>--all, -a</code>	Show all chips grouped by pin count. Conflicts with <code>--board</code> .
<code>--chip-type, -c <CHIP></code>	Show just this chip type's flash usage on the board. Conflicts with <code>--all</code> .

plugin

List available plugins from the release manifest, with versions and minimum firmware requirements. Without a connected device or `--fw-version`, minimum firmware requirements are shown for reference; with either, incompatible plugins are flagged.

```
onerom plugin
onerom plugin --all-versions
onerom plugin --type system
onerom plugin --fw-version 0.6.6
```

Option	Description
<code>--all-versions, -a</code>	Show all versions of each plugin, not just the latest.
<code>--type, -t <TYPE></code>	Filter by plugin type: system or user.
<code>--fw-version <VERSION></code>	Firmware version to check compatibility against.

Device required: no.

chips

List supported chip types — for a board, with the flash each one uses, or all grouped by pin count. Top-level alias for `firmware chips`.

```
onerom chips --board fire-24-e
onerom chips --board fire-24-e --chip-type 2364
onerom chips --all
```

Option	Description
<code>--board, -b <BOARD></code>	Show supported chips for this board. Conflicts with <code>--all</code> .
<code>--all, -a</code>	Show all chips grouped by pin count. Conflicts with <code>--board</code> .
<code>--chip-type, -c <CHIP></code>	Show just this chip type's flash usage on the board. Conflicts with <code>--all</code> .

Flash usage

For a board, each chip is listed with its **ROM size** (the chip's own capacity) and its **image size** — the flash One ROM uses to emulate it, which is often larger, and occasionally much larger. The figures, and the grouping, are the same ones published in [Chip Compatibility](#); the document is generated from the same source the CLI reads, so the two agree.

Chips are grouped by how they fit the board's socket, and the **Fit** column names the fit exactly:

Fit	Meaning
native	Chip and board have the same pin count — it goes straight in.
overhang	Chip has <i>fewer</i> pins than the board, so One ROM's top pins hang out of the socket.
larger socket (no fly-leads)	Chip has <i>more</i> pins than the board, but no address line among the extra ones: One ROM sits bottom-justified in the socket, with nothing to wire.
fly-lead to X1 / fly-lead to X1 and X2	Chip has more pins than the board, and the overhanging address line(s) must be wired to One ROM's X1 (and X2) header pin.

Every fit other than `native` is a cross-size fit, and in all of them One ROM's power pins may not line up with the socket's — **power must be rerouted to One ROM's own VCC/5V pin**. `larger socket (no fly-leads)` means no *signal* wiring is needed; it does not mean the chip simply drops in. Use `board socket` with `--chip-type` and `--gpio` to see exactly where One ROM's VCC lands.

The sizes are for a chip served alone in its slot. A banked or multi-ROM set draws further lines into the slot's address window, so its image can be larger than the figure shown here; build the firmware and run `onerom firmware inspect` to see what a specific set costs.

Chips are listed only where a size can be derived, which means Fire (RP2350) boards. An Ice (STM32) board falls back to a plain list of names. A chip type of the board's own pin count that the board cannot serve — either because no firmware serves it yet (the SRAM types, at the time of writing) or because this particular board's layout cannot place it — is named in a trailing line rather than tabulated.

Board is taken from `--board`, or inferred from a connected One ROM. `--chip-type` accepts any chip type the board can emulate, under any accepted spelling.

Example output (illustrative — your build may differ):

```
$ onerom chips --board fire-24-f
Supported chip types for fire-24-f (One ROM Fire 24 (rev F)):
```

24-pin chips (native)			
Chip	ROM size	Image size	Fit
2704	512B	512B	native
2364	8KB	8KB	native
...			

28-pin chips (with fly-leads)			
Chip	ROM size	Image size	Fit
2764	8KB	32KB	fly-lead to X1
...			

Image size is the flash One ROM uses to emulate the chip, which may exceed the chip's own ROM size. See docs/COMPATIBILITY.md.

Recognised but not servable on this board: 2016, 6116

Supported plugin types:
SystemPlugin, UserPlugin

A single chip type, on the board that makes the point — an 8KB ROM costing 256KB of flash, because One ROM overhangs a 28-pin socket to emulate a 24-pin part:

```
$ onerom chips --board fire-28-c --chip-type 2364
2364 on fire-28-c (One ROM Fire 28 (rev C)):
```

ROM size	8KB
Image size	256KB
Fit	overhang

With `--all`, chip types are listed by pin count without sizes, which are board-dependent:

```
Supported 24-pin chips:
 2016, 2316, 2332, 2364, 2704, 2708, 2716, 2732, 27C32, 28C16, 4732, 4764, ...
Supported 28-pin chips:
 231024, 23128, 23256, 23512, 23C1000, 23QL384, 23QL512, 27128, 27256, ...
Supported 32-pin chips:
 23C1001, 23C1010, 27C010, 27C020, 27C040, 29F010, 39SF010, SST39SF040, ...
Supported 40-pin chips:
 23C4100, 27C200, 27C400, 27C4100, AT27C400, HN62402, M27C400, MX23C4100, ...
```

Device required: no (a device is used only to infer the board when `--board` is omitted).

board

List supported One ROM board types, or draw a board's physical pin layouts as ASCII.

```
onerom board <COMMAND>
```

Subcommand	Purpose	Device required
<code>list</code>	List the supported board types	No
<code>header</code>	Draw a board's pin (jumper) header	No

Subcommand	Purpose	Device required
<code>socket</code>	Draw a board's ROM socket pinout	No

board list

Lists the board types, in two groups. This replaces the bare `onerom boards` of earlier releases, which no longer exists.

The first group is the boards the CLI can act on — the Fire (RP2350) boards. The second is the Ice (STM32) boards, which the CLI recognises but cannot scan, program or build firmware for; naming one on a command that needs a device or an image is an error. Commands that only report hardware still take them.

```
onerom board list
```

Example output (illustrative — your build may differ):

```
Supported One ROM board types:
  fire-24-a, fire-24-c, fire-24-d, fire-24-e, fire-24-eadb01, fire-24-f, fire-24-usb-b, fire-28-
  a, fire-28-b, fire-28-c, fire-28-d, fire-32-a, fire-32-b, fire-40-a, fire-40-b

Recognised, but not supported by the CLI:
  ice-24-d, ice-24-e, ice-24-f, ice-24-g, ice-24-i, ice-24-j, ice-24-usb-h, ice-28-a
  These boards use an STM32, rather than the RP2350 the CLI works with.
```

`onerom scan --list-boards` prints the same listing.

Device required: no.

board header

Draw a board's pin (jumper / programming) header — the 2xN header along the board's top edge — as ASCII, pad by pad. Each image-select and X pad is annotated with the MCU GPIO behind it, and on RP2350 (Fire) boards with whether that GPIO is 5V-tolerant (5V) or 3.3V-only (!!3V3!! — an ADC pin that must not be driven above 3.3V). See [Voltage Levels](#) for the ADC caveat.

```
onerom board header [--board <board>]
```

Option	Description
<code>--board, -b</code>	Board type to draw (e.g. <code>fire-24-f</code>). Inferred from a connected One ROM if omitted.

A board with no pin-header descriptor prints a short notice instead of a diagram.

```
onerom board header --board fire-24-f
```

Device required: no (a device is used only to infer `--board` when it is omitted).

board socket

Draw a board's ROM socket pinout as a DIP diagram.

```
onerom board socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
<code>--board, -b</code>	Board type to draw (e.g. <code>fire-24-f</code>). Inferred from a connected One ROM if omitted.
<code>--chip-type, -c</code>	Label pins with this chip type's ROM functions instead of GPIOs.
<code>--gpio</code>	Overlay the GPIO(s) behind each pin onto the <code>--chip-type</code> view. Requires <code>--chip-type</code> .

Without `--chip-type`, each socket pin is labelled with the GPIO(s) behind it (the GPIO map). With `--chip-type <chip>` (e.g. `2364`), the pins are labelled with that ROM's functions (address / data / chip-select / BYTE / power / ...) instead; add `--gpio` to overlay both. `--gpio` requires `--chip-type`. A pin that carries two functions on a multiplexed part (e.g. the 27C400's pin 29, `A0/D15`) shows both.

The two views that label pins with GPIOs — no `--chip-type`, or `--gpio` — need the board's GPIO map. A board without one reports that and draws nothing; the `--chip-type` function view still works, as it is drawn from the chip's pinout and the board's ROM signal assignments.

When the chip's pin count differs from the board's, the socket is drawn at the larger of the two and the smaller device is bottom-justified (see [Chip Compatibility](#)):

- emulating a **smaller** ROM on a larger One ROM, One ROM's extra pins hang out of the socket and are marked `overhang` (reroute power to One ROM's VCC/5V pin);
- emulating a **larger** ROM on a smaller One ROM, the socket pins One ROM does not reach are marked `(empty)`, and any address line there shows the One ROM `X1 / X2` header pin it must be fly-leaded to (e.g. `A12 → X1`).

In both cases One ROM's own power pins may not line up with the ROM's. With `--gpio`, the pin One ROM's VCC (or GND) lands on is annotated `(VCC) / (GND)` — e.g. `NC (VCC)` shows One ROM's VCC sitting on the ROM's NC pin — so you know where power must be applied.

With `--chip-type`, the diagram is followed by that chip's image size on this board — the flash One ROM uses to emulate it, as reported by [onerom chips](#).

`--chip-type` must be a chip type the board can emulate (native, overhang or fly-lead; see [onerom chips](#) and [Chip Compatibility](#)).

```
onerom board socket --board fire-24-f
onerom board socket --board fire-24-f --chip-type 2364
onerom board socket --board fire-24-f --chip-type 2364 --gpio
```

Device required: no (a device is used only to infer `--board` when it is omitted).

Top-level aliases

Convenience aliases for frequently used nested commands. They take the same options as their targets.

peek (top-level alias)

Alias for `inspect peek live`.

```
onerom peek live --address 0x100 --length 64
```

poke (top-level alias)

Alias for `control poke live`.

```
onerom poke live --address 0x100 --input patch.bin
```

reboot (top-level alias)

Alias for `control reboot`.

```
onerom reboot
```

ROM slot specification

Used by `--slot` in `program` and `firmware build`. Repeat `--slot` once per slot. Comma-separated `key=value` pairs:

```
file=<path_or_url>,type=<romtype>[,cs1=<logic>][,cs2=<logic>][,cs3=<logic>]
[,size-handling=<handling>][,format=<binary|ihex>][,load-address=<addr>]
[,transform=<list>]
[,cpu-freq=<freq>][,cpu-vreg=<voltage>][,led=<bool>][,force-16-bit=<bool>]
```

Key	Values / notes
file	Local path or URL to the ROM image.
type	Chip type, e.g. 2364, 2332, 2716, 27C400. Any type the target firmware can serve on the board is accepted — that is exactly what <code>chips --board</code> lists, including the overhang and fly-lead combinations (a 2764 on a Fire 24, say); see COMPATIBILITY.md . Building for firmware older than v0.7.0 accepts a narrower set, and a rejection lists what that firmware serves. Any accepted alias may be used; the exact spelling you enter is preserved in the device metadata (shown by <code>scan/inspect</code>), while the resolved type drives behaviour.
cs1, cs2, cs3	CS polarity: active-low (or 0), active-high (or 1), or ignore. The snake_case config spellings (<code>active_low</code> , <code>active_high</code>) are also accepted. Which lines are required depends on the chip type (e.g. 2332 requires cs1 and cs2). <code>ignore</code> says One ROM does not monitor the line at all — it is not a polarity, and is only permitted where the chip type or set allows it (see <code>allow_cs_ignore</code>).
size-handling (aliases <code>size</code> , <code>size_handling</code>)	<code>none</code> , <code>duplicate</code> (or <code>dup</code>), <code>truncate</code> (or <code>trunc</code>), <code>pad</code> . For an Intel HEX image, padding fills with 0xFF and duplicate is not permitted.
format	<code>binary</code> (default) or <code>ihex</code> (Intel HEX). An <code>ihex</code> file is decoded to a binary image before use; unwritten bytes read as 0xFF.
load-address (alias <code>load_address</code>)	Only valid with <code>format=ihex</code> . The absolute Intel HEX address that maps to byte 0 of the ROM, as a decimal or 0x/\$-prefixed hex value (e.g. <code>\$E000</code>). Defaults to 0.
transform	Byte-level rearrangements of the image, applied in the order given and joined with <code>+</code> . See Image transforms .

Key	Values / notes
cpu-freq	e.g. 150, 150mhz, 150MHz. Values above 150 MHz require confirmation (suppressed by <code>--yes</code>) and set overclock automatically.
cpu-vreg	e.g. 1.1, 1.10, 1.10v, 1.10V. Values above 1.10 V require confirmation (suppressed by <code>--yes</code>). Must be a supported level.
led	Boolean: on/off, true/false, 1/0.
force-16-bit (alias force_16bit)	Boolean (as above). Valid only on 40-pin boards.

Examples:

```

--slot file=kernal.bin,type=2364,cs1=active-low
--slot file=chargen.bin,type=2332,cs1=active-low,cs2=active-high
--slot file=https://example.com/basic.bin,type=2716
--slot file=small.bin,type=2364,cs1=active-low,size-handling=duplicate
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex,load-address=$E000
--slot file=kernal.bin,type=2364,cs1=active-low,cpu-freq=200MHz,cpu-vreg=1.2V
--slot file=char.bin,type=2332,cs1=active-low,cs2=active-high,led=off
--slot file=amiga.bin,type=27C400,force-16-bit=true
--slot file=undersized.bin,type=2732,size=pad
--slot file=oversized.bin,type=2732,size=trunc
--slot file=halfsize.bin,type=2732,size=dup
--slot file=amiga.bin,type=27C400,transform=swap_bytes
--slot file=rom32.bin,type=27C010,transform=deinterleave:1/2/2+swap_bytes

```

Image transforms

Some ROM images are not laid out the way the target chip needs them: a 16-bit part whose image was produced with the opposite byte order, or a wide image that interleaves several narrower devices. Transforms rearrange the bytes before the image is written into the firmware.

They are available two ways, and both run exactly the same operation:

- as the `transform=` key of a `--slot`, or a "transform" array in a config file, applied during the build;
- as the standalone `image swap-bytes` and `image deinterleave` subcommands, which rewrite a file.

Transform	Effect
swap_bytes	Reverses the byte order within each 16-bit word. The image must have an even length.
deinterleave:<offset>/<stride>	The image contains <code>stride</code> interleaved lanes one byte wide; keep lane offset.
deinterleave:<offset>/<stride>/<bytes>	As above, with lanes bytes wide — use 2 to keep 16-bit words together.

The parameters are positional, in the order `<offset>/<stride>/<bytes>`, and `<bytes>` may be omitted (it defaults to 1).

Each name has aliases, accepted identically by the CLI and by a config file:

Canonical	Also accepted
swap_bytes	swap-bytes, swapbytes
deinterleave	de_interleave, de-interleave, deint
transform= (slot key)	trans=

`deinterleave` requires the image length to be a multiple of `bytes × stride` — one full set of lanes. The result is `1/stride` of the input length.

Transforms are applied **in the order listed**, and the order matters:

```
transform=deinterleave:1/2/2+swap_bytes
```

takes the upper 16-bit half of each 32-bit word and *then* swaps its byte pairs. Note that `offset` selects which lane, not a named "high" or "low" half — which half you get depends on the byte order of your source image, and that is what `swap_bytes` is for.

Within the build pipeline, transforms run after any `location` window and after an Intel HEX image has been decoded, but before `size-handling` reconciles the image against the chip size. A `swap_bytes` on an odd-length image is an error unless `size-handling` is `pad` (which appends one blank byte) or `truncate` (which drops the trailing byte). Where the size handling is used this way it counts as having been needed, so it is not then reported as redundant even if the transformed image lands on exactly the chip size.

Common recipes:

Goal	Spec
16-bit image with the wrong byte order	transform=swap_bytes
Even / odd bytes of a 16-bit interleaved image	transform=deinterleave:0/2 / transform=deinterleave:1/2
Byte <i>n</i> of a 32-bit interleaved image	transform=deinterleave:<n>/4
One 16-bit half of a 32-bit interleaved image	transform=deinterleave:0/2/2 or deinterleave:1/2/2

The transforms applied to an image are recorded in the firmware metadata alongside its filename — as `kick.bin|transform=swap_bytes` — so a built image carries a record of how its ROM data was derived. Note that the metadata filename field is capped at 128 bytes, so the suffix can be truncated away for a very long path; use `label=` to keep it short.

Plugin specification

Used by `--plugin` in `program` and `firmware build`. At most one system plugin and one user plugin; a user plugin requires a system plugin. The system plugin is placed in slot 0, the user plugin in slot 1.

Form	Meaning
<code>--plugin usb</code>	Latest compatible version, by name.
<code>--plugin system/usb</code>	With explicit type (system or user).
<code>--plugin usb,version=0.1.0</code>	Pinned version.

Form	Meaning
<code>--plugin file=path/to/plugin.bin</code>	Local file.
<code>--plugin file=https://example.com/plugin.bin</code>	Remote file.

Pin values

Used by `--pin` in `control pin`, `control reset` and `inspect gpio`.

`--pin` names one **MCU GPIO**, either directly or through a header pad that is wired to one. All spellings are case-insensitive (`GPI023` , `SEL_A`).

Form	Meaning
<code>gpio<N></code>	An MCU GPIO — for example <code>gpio23</code> .
<code>sel_a ... sel_e</code>	An image-select pad. <code>sel-a</code> and <code>sela</code> are also accepted.
<code>x1, x2</code>	An X pad.

A pad name resolves against the **board**, since which GPIO sits behind `sel_a` is a fact about the board and not about the name. The board is normally read from the connected device; `--board` overrides it, and is what you need if this build does not recognise the device's board revision. `gpio<N>` needs no board. A board that has no such pad — `sel_e` on a four-select board, `x1` on a board with no X pads — is an error naming the pads that board does have.

Resolution uses the board's electrical pin assignments, not its header layout, so pad names work on every board, including those whose physical header is not yet characterised.

A bare number is **rejected**. `23` could be an MCU GPIO, an image-select pad, an X pad or a ROM socket pin, and driving the wrong one is not a recoverable mistake, so the CLI names the namespaces rather than guessing. Accepting pad names does not remove that ambiguity — it sharpens it.

The broken-out address pads (`a<N>`) are recognised and **deliberately refused**, now and in future. `--pin` addresses MCU GPIOs and the pads a wire can reach; an address line is a ROM signal rather than one of those, and accepting `a17` would invite `a11` or `d3`, which have no pad at all. Use the MCU GPIO behind the pad. `run`, `bootsel`, `swclk` and `swdio` are reported as not being GPIOs that can be driven. There is no syntax for a ROM socket leg.

Run `onerom inspect header` to see which GPIO is behind each header pad, or `onerom inspect gpio` for the full per-GPIO listing.

The upper bound is the device's own GPIO count — 30 on an RP2350A, 48 on an RP2350B — read from the device rather than assumed, so a GPIO the device does not have is reported against what it does have.